```
import jax
import jax.numpy as jnp
from jax import random
from jax.scipy.linalg import expm

# Enable float64 for better spectral accuracy
jax.config.update("jax_enable_x64", True)

# ------------------------------------------------------------------------
# SU(3) Lie algebra basis: T^a = i * lambda^a (Gell-Mann matrices)
# ------------------------------------------------------------------------

def su3_generators():
    """Return T[a] = i * lambda^a, a=0..7, as anti-Hermitian traceless 3x3 m
    zero = 0.0
    one  = 1.0

    lam = []

    # lambda1
    lam1 = jnp.array([[0, 1, 0],
                      [1, 0, 0],
                      [0, 0, 0]], dtype=jnp.complex128)
    lam.append(lam1)

    # lambda2
    lam2 = jnp.array([[0, -1j, 0],
                      [1j, 0, 0],
                      [0, 0, 0]], dtype=jnp.complex128)
    lam.append(lam2)

    # lambda3
    lam3 = jnp.array([[1, 0, 0],
                      [0, -1, 0],
                      [0, 0, 0]], dtype=jnp.complex128)
    lam.append(lam3)

    # lambda4
    lam4 = jnp.array([[0, 0, 1],
                      [0, 0, 0],
                      [1, 0, 0]], dtype=jnp.complex128)
    lam.append(lam4)

    # lambda5
    lam5 = jnp.array([[0, 0, -1j],
                      [0, 0, 0],
                      [1j, 0, 0]], dtype=jnp.complex128)
    lam.append(lam5)
```

```
    # lambda6
    lam6 = jnp.array([[0, 0, 0],
                      [0, 0, 1],
                      [0, 1, 0]], dtype=jnp.complex128)
    lam.append(lam6)

    # lambda7
    lam7 = jnp.array([[0, 0, 0],
                      [0, 0, -1j],
                      [0, 1j, 0]], dtype=jnp.complex128)
    lam.append(lam7)

    # lambda8
    lam8 = (1.0 / jnp.sqrt(3.0)) * jnp.array([[1, 0, 0],
                                              [0, 1, 0],
                                              [0, 0, -2]], dtype=jnp.complex
    lam.append(lam8)

    lam = jnp.stack(lam, axis=0)         # (8,3,3)
    T = 1j * lam                         # T^a = i lambda^a, anti-Hermitian
    return T


# ----------------------------------------------------------------------------
# Theta -> A(x,mu) -> U(x,mu) on a 4D L^4 lattice with 4 directions
# ----------------------------------------------------------------------------

def theta_to_A(theta, L, T):
    """
    theta: flat vector of shape (L^4 * 4 * 8,) with real components.
    Returns A[t,x,y,z,mu,:,:] as 3x3 anti-Hermitian matrices.
    """
    theta = theta.reshape(L, L, L, L, 4, 8)  # (t,x,y,z,mu,a)
    # Contract over generator index a: sum_a theta * T[a]
    # Result: (t,x,y,z,mu,3,3)
    A = jnp.tensordot(theta, T, axes=([5], [0]))
    return A


def A_to_U(A):
    """
    A: (...,3,3) anti-Hermitian matrices
    Returns U = exp(A) with same leading shape.
    """
    # Flatten leading dims, vmap expm, then reshape back
    leading_shape = A.shape[:-2]
    A_flat = A.reshape((-1, 3, 3))

    U_flat = jax.vmap(expm)(A_flat)
    U = U_flat.reshape(leading_shape + (3, 3))
    return U
```

```
    return 0


# ---------------------------------------------------------------------------
# Wilson action and Haar mass term
# ---------------------------------------------------------------------------

def wilson_action(theta, L, beta, T):
    """
    Standard Wilson action for SU(3) on a periodic 4D L^4 lattice.
    theta: flat (L^4 * 4 * 8,) real vector.
    """
    A = theta_to_A(theta, L, T)          # (t,x,y,z,mu,3,3)
    U = A_to_U(A)

    def idx(i):
        return i % L

    S = 0.0
    # Directions mu,nu = 0..3
    for mu in range(4):
        for nu in range(mu + 1, 4):
            for t in range(L):
                for x in range(L):
                    for y in range(L):
                        for z in range(L):
                            # coordinates
                            t1 = idx(t + (mu == 0))
                            x1 = idx(x + (mu == 1))
                            y1 = idx(y + (mu == 2))
                            z1 = idx(z + (mu == 3))

                            t2 = idx(t + (nu == 0))
                            x2 = idx(x + (nu == 1))
                            y2 = idx(y + (nu == 2))
                            z2 = idx(z + (nu == 3))

                            t3 = idx(t + (mu == 0) + (nu == 0))
                            x3 = idx(x + (mu == 1) + (nu == 1))
                            y3 = idx(y + (mu == 2) + (nu == 2))
                            z3 = idx(z + (mu == 3) + (nu == 3))

                            U_mu = U[t, x, y, z, mu]        # U(x,mu)
                            U_nu = U[t1, x1, y1, z1, nu]    # U(x+mu,nu)
                            U_mu_dag = jnp.conjugate(U[t2, x2, y2, z2, mu].T
                            U_nu_dag = jnp.conjugate(U[t, x, y, z, nu].T)

                            U_p = U_mu @ U_nu @ U_mu_dag @ U_nu_dag
                            tr_p = jnp.trace(U_p)
                            S = S + (1.0 - (1.0 / 3.0) * jnp.real(tr_p))
    return beta * S
```

```
def haar_action(theta, L, c0, T):
    """
    Quadratic Haar mass term S_Haar = c0 * sum_links ||A||_F^2.
    """
    A = theta_to_A(theta, L, T)          # (t,x,y,z,mu,3,3)
    # Frobenius norm squared per link
    norm_sq = jnp.real(jnp.vdot(A, A))   # sum over all entries |A|^2
    return c0 * norm_sq


def total_action(theta, L, beta, c0, T):
    return wilson_action(theta, L, beta, T) + haar_action(theta, L, c0, T)


# ---------------------------------------------------------------------------
# Hessian and eigenvalues
# ---------------------------------------------------------------------------

def hessian_total(theta0, L, beta, c0, T):
    """
    Hessian of S_total at theta0.
    Returns a (dim, dim) symmetric matrix.
    """
    def f(th):
        return total_action(th, L, beta, c0, T)

    H = jax.hessian(f)(theta0)
    return H


def hessian_wilson(theta0, L, beta, T):
    def f(th):
        return wilson_action(th, L, beta, T)
    return jax.hessian(f)(theta0)


# ---------------------------------------------------------------------------
# Random-direction scan
# ---------------------------------------------------------------------------

def random_theta(key, L, scale):
    """
    Sample a random theta with given amplitude scale.
    """
    dim = (L ** 4) * 4 * 8
    key, sub = random.split(key)
    # Normal(0,1) then rescale
    th = scale * random.normal(sub, shape=(dim,), dtype=jnp.float64)
    return th, key
```

```
def scan_random_directions(L=2, beta=1.0, c0=0.25,
                           scales=(0.1, 0.2, 0.3),
                           n_samples=5,
                           seed=0):
    """
    For each scale and n_samples, compute min eigenvalues of
    Hessian(Wilson) and Hessian(Wilson+Haar).
    Returns a dict of results.
    """
    T = su3_generators()
    dim = (L ** 4) * 4 * 8
    key = random.PRNGKey(seed)

    results = []
    for scale in scales:
        lamW_min_list = []
        lamWH_min_list = []
        for _ in range(n_samples):
            theta, key = random_theta(key, L, scale)
            Hw = hessian_wilson(theta, L, beta, T)
            Hwh = hessian_total(theta, L, beta, c0, T)

            # Eigenvalues (symmetric matrices)
            ew = jnp.linalg.eigvalsh(Hw)
            ewh = jnp.linalg.eigvalsh(Hwh)

            lamW_min_list.append(jnp.min(ew))
            lamWH_min_list.append(jnp.min(ewh))

        lamW_min = float(jnp.min(jnp.array(lamW_min_list)))
        lamW_max = float(jnp.max(jnp.array(lamW_min_list)))
        lamWH_min = float(jnp.min(jnp.array(lamWH_min_list)))
        lamWH_max = float(jnp.max(jnp.array(lamWH_min_list)))

        results.append({
            "scale": scale,
            "lamW_min_min": lamW_min,
            "lamW_min_max": lamW_max,
            "lamWH_min_min": lamWH_min,
            "lamWH_min_max": lamWH_max,
        })

    return results


# ------------------------------------------------------------------------
# Main: sanity check at theta=0 and a small random scan
# ------------------------------------------------------------------------
```

```
if __name__ == "__main__":
    L = 2
    beta = 1.0
    c0 = 0.25  # SU(3) Haar mass coefficient from analytic expansion

    T = su3_generators()
    dim = (L ** 4) * 4 * 8
    theta0 = jnp.zeros((dim,), dtype=jnp.float64)

    # Hessians at the origin
    print("Computing Hessians at theta=0 ...")
    H_haar = jax.hessian(lambda th: haar_action(th, L, c0, T))(theta0)
    H_wilson = hessian_wilson(theta0, L, beta, T)
    H_total = hessian_total(theta0, L, beta, c0, T)

    e_haar = jnp.linalg.eigvalsh(H_haar)
    e_wilson = jnp.linalg.eigvalsh(H_wilson)
    e_total = jnp.linalg.eigvalsh(H_total)

    print("\n=== Eigenvalues at theta=0 ===")
    print(f"Haar-only:    lambda_min = {float(jnp.min(e_haar)):.9f}, "
          f"lambda_max = {float(jnp.max(e_haar)):.9f}")
    print(f"Wilson-only:  lambda_min = {float(jnp.min(e_wilson)):.9e}, "
          f"lambda_max = {float(jnp.max(e_wilson)):.9f}")
    print(f"Total (W+H):  lambda_min = {float(jnp.min(e_total)):.9f}, "
          f"lambda_max = {float(jnp.max(e_total)):.9f}")

    # Small random-direction scan
    print("\nScanning random directions ...")
    res = scan_random_directions(
        L=L,
        beta=beta,
        c0=c0,
        scales=(0.1, 0.2, 0.3),
        n_samples=3,
        seed=0,
    )

    print("\n=== Random-direction min eigenvalues ===")
    print("scale   lamW_min[min,max]      lamW+H_min[min,max]")
    for r in res:
        print(f"{r['scale']:.3f}   "
              f"{r['lamW_min_min']:.6f}, {r['lamW_min_max']:.6f}   "
              f"{r['lamWH_min_min']:.6f}, {r['lamWH_min_max']:.6f}")
```

```
Computing Hessians at theta=0 ...
/usr/local/lib/python3.12/dist-packages/jax/_src/lax/lax.py:5473: ComplexWarn
  x_bar = _convert_element_type(x_bar, x.aval.dtype, x.aval.weak_type)
```

```
=== Eigenvalues at theta=0 ===
Haar-only:    lambda_min = 1.000000000, lambda_max = 1.000000000
Wilson-only:  lambda_min = -5.692186700e-15, lambda_max = 10.666666667
Total (W+H):  lambda_min = 1.000000000, lambda_max = 11.666666667

Scanning random directions ...

=== Random-direction min eigenvalues ===
scale   lamW_min[min,max]      lamW+H_min[min,max]
0.100   -1.137269, -1.019141   -0.137269, -0.019141
0.200   -2.419104, -2.333659   -1.419104, -1.333659
0.300   -3.341805, -2.843694   -2.341805, -1.843694
```

```
# ======================================================================
# SU(3) WILSON-ONLY OPTIMIZED DIRECTIONAL HESSIAN BOUND ENGINE (L=4)
# ======================================================================
# Finds worst-case Wilson curvature constant C_W:
#
#   C_W*(theta) = max_{||v||=1}  -lambda_min( Hess_W(theta * v) ) / theta^2
#
# Uses:
#   • Padé[2/2] SU(3) exponential
#   • Wilson-only action
#   • Hessian-vector products (JVP)
#   • Lanczos extremal eigenvalue solver
#   • Projected gradient ascent on the unit sphere
#
# ======================================================================

import jax
import jax.numpy as jnp
import numpy as np
from jax import lax
jax.config.update("jax_enable_x64", False)   # FP32 = faster

# ======================================================================
# 1. SU(3) GENERATORS
# ======================================================================

def su3_generators():
    lam1 = jnp.array([[0,1,0],[1,0,0],[0,0,0]], jnp.complex64)
    lam2 = jnp.array([[0,-1j,0],[1j,0,0],[0,0,0]], jnp.complex64)
    lam3 = jnp.array([[1,0,0],[0,-1,0],[0,0,0]], jnp.complex64)
    lam4 = jnp.array([[0,0,1],[0,0,0],[1,0,0]], jnp.complex64)
    lam5 = jnp.array([[0,0,-1j],[0,0,0],[1j,0,0]], jnp.complex64)
    lam6 = jnp.array([[0,0,0],[0,0,1],[0,1,0]], jnp.complex64)
    lam7 = jnp.array([[0,0,0],[0,0,-1j],[0,1j,0]], jnp.complex64)
    lam8 = jnp.array([[1,0,0],[0,1,0],[0,0,-2]], jnp.complex64) / jnp.sqrt(3
    lam = jnp.stack([lam1,lam2,lam3,lam4,lam5,lam6,lam7,lam8], 0)
    return 1j * lam / 2.0
```

```
T_SU3 = su3_generators()

def su3_alg_from_vec(a):
    return jnp.einsum("...a,aij->...ij", a, T_SU3)

# ============================================================================
# 2. Padé [2/2] SU(3) exponential
# ============================================================================

def su3_exp_pade22(A):
    I = jnp.eye(3, dtype=jnp.complex64)
    A2 = A @ A
    c1 = 0.5
    c2 = 1/12.0
    Num = I + c1*A + c2*A2
    Den = I - c1*A + c2*A2
    return jnp.linalg.solve(Den, Num)

# ============================================================================
# 3. Build SU(3) links
# ============================================================================

def build_links_factory(L):
    def build_links(params):
        flat = params.reshape(-1, 8)
        A = jax.vmap(su3_alg_from_vec)(flat)
        U = jax.vmap(su3_exp_pade22)(A)
        return U.reshape(L, L, L, L, 4, 3, 3)
    return build_links

# ============================================================================
# 4. Wilson action (vectorized over lattice)
# ============================================================================

def wilson_action(params, L, beta, build_links_L):
    U = build_links_L(params)
    S = 0.0
    for mu in range(4):
        for nu in range(mu+1,4):
            U_mu = U[..., mu, :, :]
            U_nu_shift = jnp.roll(U[..., nu, :, :], -1, axis=mu)
            U_mu_dag_shift = jnp.swapaxes(
                jnp.conjugate(jnp.roll(U[..., mu, :, :], -1, axis=nu)),
                -1, -2
            )
            U_nu_dag = jnp.swapaxes(jnp.conjugate(U[..., nu, :, :]), -1, -2)
            P = U_mu @ U_nu_shift @ U_mu_dag_shift @ U_nu_dag
            trP = jnp.real(jnp.einsum("...ii->...", P))
            S += jnp.sum(1.0 - trP/3.0)
    return beta * S
```

```
# ========================================================================
# 5. Flat interface for Hessian-vector products
# ========================================================================

def make_flat_funcs(L, beta):
    build_links_L = build_links_factory(L)

    def unflatten(x):
        return x.reshape((L, L, L, L, 4, 8))

    def flat_action(theta):
        params = unflatten(theta)
        return wilson_action(params, L, beta, build_links_L)

    return jax.jit(flat_action), (L**4 * 4 * 8)

# ========================================================================
# 6. hvp: H * v (Hessian-vector product)
# ========================================================================

def hvp(flat_action, theta, v):
    g = jax.grad(flat_action)
    _, hv = jax.jvp(g, (theta,), (v,))
    return hv

# ========================================================================
# 7. Lanczos min-eigenvalue extraction
# ========================================================================

def lanczos_min(flat_action, theta, k=20, seed=0):
    key = jax.random.PRNGKey(seed)
    n = theta.shape[0]
    v0 = jax.random.normal(key, (n,))
    v0 /= jnp.linalg.norm(v0)

    def step(carry, _):
        v_prev, v, beta_prev = carry
        w = hvp(flat_action, theta, v)
        w -= beta_prev * v_prev
        alpha = jnp.dot(w, v)
        w -= alpha * v
        beta = jnp.linalg.norm(w)
        v_next = w / (beta + 1e-8)
        return (v, v_next, beta), (alpha, beta)

    (_, _, _), (a, b) = lax.scan(step,
                                 init=(jnp.zeros_like(v0), v0, 0.0),
                                 xs=None,
                                 length=k)
    a = jnp.array(a)
```

```
    b = jnp.array(b[:-1])
    T = jnp.diag(a) + jnp.diag(b,1) + jnp.diag(b,-1)
    eigs = jnp.linalg.eigvalsh(T)
    return float(eigs[0])

# ============================================================================
# 8. Optimized direction search on the sphere
# ============================================================================

def sphere_project(x):
    return x / jnp.linalg.norm(x)

def optimize_direction(flat_action, theta, n_iter=25, lr=5e-3, seed=0):
    """
    Maximize:  C_W(v) = -lambda_min(H(theta*v)) / theta^2
    over ||v||=1 using projected gradient ascent.
    """

    key = jax.random.PRNGKey(seed)
    n = theta.shape[0]

    # initialize v uniformly on sphere
    v = jax.random.normal(key, (n,))
    v = sphere_project(v)

    def C_W(v):
        lam = lanczos_min(flat_action, theta*v, k=20, seed=seed)
        return -lam / (jnp.dot(theta*v, theta*v))  # = -lam / theta^2

    grad_C = jax.grad(lambda v: C_W(v))

    for it in range(n_iter):
        g = grad_C(v)
        v = v + lr * g
        v = sphere_project(v)

    return v, float(C_W(v))

# ============================================================================
# 9. Sweep over amplitudes θ for L = 4
# ============================================================================

def sweep_theta_optimized(L=4,
                          beta=1.0,
                          thetas=[0.02, 0.04, 0.06, 0.08, 0.10],
                          n_iter=25,
                          lr=5e-3):

    flat_action, n_params = make_flat_funcs(L, beta)
    results = []
    print(f"L={L}, n_params={n_params}")
```

```
    for t in thetas:
        theta_vec = t * jnp.ones((n_params,), dtype=jnp.float32)  # scale fa
        v_opt, C_opt = optimize_direction(flat_action,
                                          theta_vec,
                                          n_iter=n_iter,
                                          lr=lr,
                                          seed=int(1e6*t))

        print(f"θ={t:.3f}  C_W*(θ) = {C_opt:.6f}")
        results.append((t, C_opt, v_opt))

    return results

# ============================================================================
# RUN ENGINE
# ============================================================================

print("=== SU(3) Wilson-only Optimized Direction Bound (L=4) ===")
res = sweep_theta_optimized(L=4, beta=1.0)
print("\nDone.")
```

Next steps: Explain error

```
# ==============================================================================
# HIGH-PRECISION A100 SU(3) CONVEXITY ENGINE (L=12 HORIZON CHECK)
# ==============================================================================
# Features:
# 1. JAX Double Precision (x64) enabled to distinguish physical mass from no
# 2. L=12 Volume to rule out finite-size artifacts in the Gribov Horizon.
# 3. Checkpointed Matrix-Free formulation to fit L=12 in GPU VRAM.
# ==============================================================================

import jax
import jax.numpy as jnp
import numpy as np
import jax.lax as lax
import time

# 1. FORCE DOUBLE PRECISION
jax.config.update("jax_enable_x64", True)

# ==============================================================================
# CONSTANTS & ALGEBRA
# ==============================================================================
C0_SU3 = 3.0 / 24.0  # Correct Haar coefficient = 0.125

def su3_generators():
    """Return the standard anti-Hermitian SU(3) generators T_a = i lambda_a
    # Explicit complex128 for precision
    lam1 = jnp.array([[0,1,0],[1,0,0],[0,0,0]], dtype=jnp.complex128)
    lam2 = jnp.array([[0,-1j,0],[1j,0,0],[0,0,0]], dtype=jnp.complex128)
    lam3 = jnp.array([[1,0,0],[0,-1,0],[0,0,0]], dtype=jnp.complex128)
    lam4 = jnp.array([[0,0,1],[0,0,0],[1,0,0]], dtype=jnp.complex128)
    lam5 = jnp.array([[0,0,-1j],[0,0,0],[1j,0,0]], dtype=jnp.complex128)
    lam6 = jnp.array([[0,0,0],[0,0,1],[0,1,0]], dtype=jnp.complex128)
    lam7 = jnp.array([[0,0,0],[0,0,-1j],[0,1j,0]], dtype=jnp.complex128)
    lam8 = jnp.array([[1,0,0],[0,1,0],[0,0,-2]], dtype=jnp.complex128) / jnp

    lam = jnp.stack([lam1,lam2,lam3,lam4,lam5,lam6,lam7,lam8], axis=0)
    return 1j * lam / 2.0

T_SU3 = su3_generators()

def su3_alg_from_vec(a):
    """Map R^8 -> su(3) matrix using generators."""
    return jnp.einsum("...a,aij->...ij", a, T_SU3)

@jax.checkpoint
def su3_exp_pade22(A):
    """
```

```
    Padé [2/2] approximant for exp(A).
    Accurate and stable for ||A|| < ~1.
    Fully differentiable and checkpointed for memory efficiency.
    """
    I = jnp.eye(3, dtype=jnp.complex128)
    A2 = A @ A
    c1 = 0.5
    c2 = 1.0/12.0
    Num = I + c1*A + c2*A2
    Den = I - c1*A + c2*A2
    return jnp.linalg.solve(Den, Num)

# ============================================================================
# LATTICE & ACTION
# ============================================================================
def build_links_factory(L):
    @jax.checkpoint
    def build_links(params):
        flat = params.reshape(-1, 8)
        A = jax.vmap(su3_alg_from_vec)(flat)
        U = jax.vmap(su3_exp_pade22)(A)
        return U.reshape(L, L, L, L, 4, 3, 3)
    return build_links

def compute_plaquette_sum(U, beta):
    """Compute Wilson Action (without checkpointing for speed)."""
    S = 0.0
    for mu in range(4):
        for nu in range(mu+1, 4):
            U_mu = U[..., mu, :, :]
            U_nu_shift = jnp.roll(U[..., nu, :, :], -1, axis=mu)
            U_mu_dag_shift = jnp.swapaxes(
                jnp.conjugate(jnp.roll(U[..., mu, :, :], -1, axis=nu)), -1,
            )
            U_nu_dag = jnp.swapaxes(jnp.conjugate(U[..., nu, :, :]), -1, -2)

            P = U_mu @ U_nu_shift @ U_mu_dag_shift @ U_nu_dag
            trP = jnp.real(jnp.einsum("...ii->...", P))
            S += jnp.sum(1.0 - trP/3.0)
    return beta * S

def vectorized_wilson_action(params, L, beta, build_links_L):
    U = build_links_L(params)
    return compute_plaquette_sum(U, beta)

def haar_mass(params, c0):
    """Quadratic Haar Mass term: c0 * Tr(A^dag A)."""
    flat = params.reshape(-1, 8)
    def per(a):
        A = su3_alg_from_vec(a)
        return jnp.real(jnp.trace(A.conj().T @ A))
```

```
        return jnp.real(jnp.trace(A.conj().T @ A))
    return c0 * jax.vmap(per)(flat).sum()

def make_flat_funcs(L, beta, c0=C0_SU3):
    build_links_L = build_links_factory(L)

    def unflatten(theta):
        return theta.reshape((L, L, L, L, 4, 8))

    def flat_action(theta):
        params = unflatten(theta)
        return (
            vectorized_wilson_action(params, L, beta, build_links_L)
            + haar_mass(params, c0)
        )
    # Return JIT-compiled function and param count
    return jax.jit(flat_action), (L**4 * 4 * 8)

# ============================================================================
# HESSIAN VECTOR PRODUCT & LANCZOS
# ============================================================================
def hvp(flat_action, theta, v):
    """Matrix-free Hessian-Vector Product via Forward-over-Reverse AD."""
    g = jax.grad(flat_action)
    _, hv = jax.jvp(g, (theta,), (v,))
    return hv

def lanczos_min(flat_action, theta, k=25, seed=0):
    """
    Estimate min eigenvalue using Lanczos iteration.
    k=25 is sufficient for extreme eigenvalues.
    """
    key = jax.random.PRNGKey(seed)
    n = theta.shape[0]

    # Random start vector
    v0 = jax.random.normal(key, (n,), dtype=jnp.float64)
    v0 /= jnp.linalg.norm(v0)

    def step(carry, _):
        v_prev, v, beta_prev = carry

        # Apply Hessian
        w = hvp(flat_action, theta, v)

        # Orthogonalize
        w = w - beta_prev * v_prev
        alpha = jnp.dot(w, v)
        w = w - alpha * v

        beta = jnp.linalg.norm(w)
```

```
            v_next = w / (beta + 1e-12) # higher stability epsilon for x64
            return (v, v_next, beta), (alpha, beta)

        _, (alphas, betas) = lax.scan(
            step, init=(jnp.zeros_like(v0), v0, 0.0), xs=None, length=k
        )

        # Build Tridiagonal Matrix T
        alphas = jnp.array(alphas)
        betas = jnp.array(betas[:-1])
        T = jnp.diag(alphas) + jnp.diag(betas, 1) + jnp.diag(betas, -1)

        # Diagonalize small T matrix
        eigs = jnp.linalg.eigvalsh(T)
        return float(eigs[0])

    # ==============================================================================
    # EXECUTION: HORIZON CHECK SCAN
    # ==============================================================================
    def sample_theta(L, scale, key):
        # Ensure float64 sampling
        return (scale * jax.random.normal(key, (L, L, L, L, 4, 8), dtype=jnp.flo

    def convexity_grid(L, betas, scales, c0=C0_SU3, n_samples=2, seed=42):
        key = jax.random.PRNGKey(seed)
        print(f"\n=== High-Precision Gribov Check L={L}, Haar={c0} ===")
        print(f"Precision: {jnp.zeros(1).dtype}")
        print(f"Parameter Count: {L**4 * 32:,}")

        start_time = time.time()

        results = []

        for beta in betas:
            # Recompile for new beta
            flat_action, _ = make_flat_funcs(L, float(beta), c0)

            for scale in scales:
                lam_vals = []
                for i in range(n_samples):
                    key, sub = jax.random.split(key)
                    theta = sample_theta(L, scale, sub)

                    # Different seed for Lanczos start vector
                    key, sub2 = jax.random.split(key)
                    lam = lanczos_min(flat_action, theta, k=25, seed=int(sub2[0]
                    lam_vals.append(lam)

                # We take the worst case (minimum curvature found)
                min_lam = np.min(lam_vals)
                results.append((beta, scale, min_lam))
```

```
            results.append((beta, scale, min_lam))

            status = "CONVEX" if min_lam > 0 else "UNSTABLE"
            print(f"beta={beta:4.2f} scale={scale:5.3f} lam={min_lam:+.6f} [

    print(f"Total Time: {time.time()-start_time:.2f}s")
    return results

# --- RUN CONFIGURATION ---
# Targeted Scan: Weak coupling region where L=8 showed shrinking horizon.
# If L=12 shows positive lambda here, the shrinkage is physical (Gribov), no
betas = jnp.linspace(2.0, 3.0, 4)  # 2.0, 2.33, 2.66, 3.0
scales = (0.04, 0.06)              # Narrow search around the horizon edge

# Launch
if __name__ == "__main__":
    res = convexity_grid(L=12, betas=betas, scales=scales, n_samples=2)
```

```
=== High-Precision Gribov Check L=12, Haar=0.125 ===
Precision: float64
Parameter Count: 663,552
beta=2.00 scale=0.040 lam=+0.059935 [CONVEX]
beta=2.00 scale=0.060 lam=+0.018378 [CONVEX]
beta=2.33 scale=0.040 lam=+0.049282 [CONVEX]
beta=2.33 scale=0.060 lam=+0.000649 [CONVEX]
beta=2.67 scale=0.040 lam=+0.038355 [CONVEX]
beta=2.67 scale=0.060 lam=-0.017361 [UNSTABLE]
beta=3.00 scale=0.040 lam=+0.027416 [CONVEX]
beta=3.00 scale=0.060 lam=-0.035343 [UNSTABLE]
Total Time: 79.53s
```

```
import jax
import jax.numpy as jnp
import numpy as np
from jax import lax

jax.config.update("jax_enable_x64", False)  # FP32 is fine here

# ============================================================
# 1. SU(3) generators: T_a = i λ_a / 2 (anti-Hermitian)
# ============================================================
def su3_generators():
    lam1 = jnp.array([[0,1,0],[1,0,0],[0,0,0]], jnp.complex64)
    lam2 = jnp.array([[0,-1j,0],[1j,0,0],[0,0,0]], jnp.complex64)
    lam3 = jnp.array([[1,0,0],[0,-1,0],[0,0,0]], jnp.complex64)
    lam4 = jnp.array([[0,0,1],[0,0,0],[1,0,0]], jnp.complex64)
    lam5 = jnp.array([[0,0,-1j],[0,0,0],[1j,0,0]], jnp.complex64)
    lam6 = jnp.array([[0,0,0],[0,0,1],[0,1,0]], jnp.complex64)
    lam7 = jnp.array([[0,0,0],[0,0,-1j],[0,1j,0]], jnp.complex64)
    lam8 = jnp.array([[1,0,0],[0,1,0],[0,0,-2]], jnp.complex64) / jnp.sqrt(3)
    lam = jnp.stack([lam1,lam2,lam3,lam4,lam5,lam6,lam7,lam8], 0)
```

```
    return 1j * lam / 2.0

T_SU3 = su3_generators()

def su3_alg_from_vec(a):
    return jnp.einsum("...a,aij->...ij", a, T_SU3)

# ============================================================
# 2. Padé [2/2] SU(3) exponential
# ============================================================
def su3_exp_pade22(A):
    I = jnp.eye(3, dtype=jnp.complex64)
    A2 = A @ A
    c1 = 0.5
    c2 = 1/12.0
    Num = I + c1*A + c2*A2
    Den = I - c1*A + c2*A2
    return jnp.linalg.solve(Den, Num)

# ============================================================
# 3. Build links from θ (A-coordinates)
# ============================================================
def build_links_factory(L):
    def build_links(params):
        flat = params.reshape(-1, 8)
        A = jax.vmap(su3_alg_from_vec)(flat)
        U = jax.vmap(su3_exp_pade22)(A)
        return U.reshape(L, L, L, L, 4, 3, 3)
    return build_links

# ============================================================
# 4. Wilson action (SU(3))
# ============================================================
def wilson_action(params, L, beta, build_links_L):
    U = build_links_L(params)
    S = 0.0
    for mu in range(4):
        for nu in range(mu+1, 4):
            U_mu = U[..., mu, :, :]
            U_nu_shift = jnp.roll(U[..., nu, :, :], -1, axis=mu)
            U_mu_dag_shift = jnp.swapaxes(
                jnp.conjugate(jnp.roll(U[..., mu, :, :], -1, axis=nu)),
                -1, -2
            )
            U_nu_dag = jnp.swapaxes(jnp.conjugate(U[..., nu, :, :]), -1, -2)
            P = U_mu @ U_nu_shift @ U_mu_dag_shift @ U_nu_dag
            trP = jnp.real(jnp.einsum("...ii->...", P))
            S += jnp.sum(1.0 - trP/3.0)
    return beta * S
```

```
# ============================================================
# 5. Flat action and hvp
# ============================================================
def make_flat_funcs(L, beta):
    build_links_L = build_links_factory(L)

    def unflatten(theta):
        return theta.reshape((L, L, L, L, 4, 8))

    def flat_action(theta):
        params = unflatten(theta)
        return wilson_action(params, L, beta, build_links_L)

    return jax.jit(flat_action), (L**4 * 4 * 8)

def hvp(flat_action, theta, v):
    g = jax.grad(flat_action)
    _, hv = jax.jvp(g, (theta,), (v,))
    return hv

# ============================================================
# 6. Lanczos min eigenvalue (no JAX grad through here)
# ============================================================
def lanczos_min(flat_action, theta, k=20, seed=0):
    key = jax.random.PRNGKey(seed)
    n = theta.shape[0]
    v0 = jax.random.normal(key, (n,))
    v0 = v0 / jnp.linalg.norm(v0)

    def step(carry, _):
        v_prev, v, beta_prev = carry
        w = hvp(flat_action, theta, v)
        w = w - beta_prev * v_prev
        alpha = jnp.dot(w, v)
        w = w - alpha * v
        beta = jnp.linalg.norm(w)
        v_next = w / (beta + 1e-8)
        return (v, v_next, beta), (alpha, beta)

    (_, _, _), (a, b) = lax.scan(
        step,
        init=(jnp.zeros_like(v0), v0, 0.0),
        xs=None,
        length=k
    )
    a = jnp.array(a)
    b = jnp.array(b[:-1])
    T = jnp.diag(a) + jnp.diag(b, 1) + jnp.diag(b, -1)
    eigs = jnp.linalg.eigvalsh(T)
    # Important: do NOT wrap in float() if this is ever traced;
    # here we're outside jax.grad so it's safe, but keep it as array:
```

```
    return jnp.min(eigs)

# ============================================================
# 7. Utility: random unit direction, C_W(theta, v)
# ============================================================
def random_unit_direction(key, n):
    v = jax.random.normal(key, (n,))
    return v / jnp.linalg.norm(v)

def C_W_value(flat_action, theta_vec, v, lanczos_seed=0):
    # theta_vec: shape (n_params,), amplitude encoded in its norm
    lam_min = lanczos_min(flat_action, theta_vec * v, k=20, seed=lanczos_seed)
    lam_min = float(lam_min)  # convert to Python float; no grad here
    theta2 = float(jnp.dot(theta_vec, theta_vec))
    return -lam_min / theta2

# ============================================================
# 8. Derivative-free hill-climbing on the sphere
# ============================================================
def optimize_direction_derivative_free(flat_action,
                                       theta_vec,
                                       n_outer=10,
                                       n_perturb=6,
                                       step_size=0.2,
                                       seed=0):
    """
    Hill-climb on v ∈ S^{n-1} to maximize C_W(theta_vec, v).
    Uses random perturbations and accepts improvements.
    """

    key = jax.random.PRNGKey(seed)
    n = theta_vec.shape[0]

    # initial random direction
    key, sub = jax.random.split(key)
    v = random_unit_direction(sub, n)

    best_C = C_W_value(flat_action, theta_vec, v, lanczos_seed=0)
    best_v = v

    for it in range(n_outer):
        for j in range(n_perturb):
            key, sub = jax.random.split(key)
            delta = random_unit_direction(sub, n)
            trial_v = best_v + step_size * delta
            trial_v = trial_v / jnp.linalg.norm(trial_v)

            C_trial = C_W_value(flat_action, theta_vec, trial_v,
                                lanczos_seed=it * n_perturb + j)
```

```
            if C_trial > best_C:
                best_C = C_trial
                best_v = trial_v

        # optional: decay step_size to refine
        step_size *= 0.7

    return best_v, best_C

# ============================================================
# 9. θ sweep for L=4
# ============================================================
def sweep_theta_optimized(L=4,
                          beta=1.0,
                          thetas=(0.02, 0.04, 0.06, 0.08, 0.10),
                          n_outer=8,
                          n_perturb=5,
                          step_size=0.3):

    flat_action, n_params = make_flat_funcs(L, beta)
    print(f"L={L}, n_params={n_params}")
    results = []

    for t in thetas:
        theta_vec = jnp.full((n_params,), t, dtype=jnp.float32)

        v_opt, C_opt = optimize_direction_derivative_free(
            flat_action,
            theta_vec,
            n_outer=n_outer,
            n_perturb=n_perturb,
            step_size=step_size,
            seed=int(1e6 * t)
        )

        print(f"θ={t:.3f}  approx C_W*(θ) = {C_opt:.6f}")
        results.append((t, C_opt, v_opt))

    return results

# ============================================================
# RUN
# ============================================================
print("=== SU(3) Wilson-only Optimized Direction Bound (L=4, hill-climb) ===")
res = sweep_theta_optimized(L=4,
                            beta=1.0,
                            thetas=(0.02, 0.04, 0.06),
                            n_outer=6,
                            n_perturb=4,
                            step_size=0.3)
print("\nDone.")
```

```
=== SU(3) Wilson-only Optimized Direction Bound (L=4, hill-climb) ===
L=4, n_params=8192
θ=0.020  approx C_W*(θ) = 0.000086
θ=0.040  approx C_W*(θ) = 0.000027
θ=0.060  approx C_W*(θ) = 0.000016

Done.
```

Start coding or generate with AI.